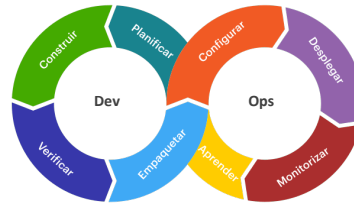


INTRODUCCIÓN A DOCKER

□ Filosofía DevOps

□ Principios DevOps

- Colaboración
- Integración continua (CI)
- Entrega continua (CD)
- Automatización
- Monitoreo y retroalimentación
- Mejora continua



Introducción a Docker

Antes de estudiar Docker es necesario comprender el contexto en el que surgió y las necesidades que impulsaron su aparición. Docker no nació de manera aislada, sino como una respuesta a problemas recurrentes que durante muchos años afectaron el desarrollo y la operación de sistemas informáticos.

Tradicionalmente, en las organizaciones existían dos áreas claramente diferenciadas. Por un lado se encontraban los desarrolladores, responsables de diseñar y programar las aplicaciones. Por otro lado, estaban los equipos de operaciones o infraestructura, encargados de administrar servidores, sistemas operativos y entornos de producción. Esta separación generaba frecuentes inconvenientes cuando una aplicación pasaba de la etapa de desarrollo a la de implementación.

Era habitual que una aplicación funcionara correctamente en la computadora del desarrollador, pero presentara errores al ejecutarse en el servidor donde debía ponerse en producción. Las causas podían ser muy diversas: diferencias de configuración, versiones incompatibles de software, dependencias faltantes, bibliotecas instaladas de manera distinta o incluso características específicas del sistema operativo utilizado. Como consecuencia, se producían retrasos, conflictos entre equipos y dificultades para garantizar la estabilidad de los sistemas.

Para resolver estos problemas surgió la filosofía DevOps. Su objetivo principal es eliminar las barreras tradicionales entre desarrollo y operaciones, promoviendo una forma de trabajo colaborativa en la que ambos equipos participen conjuntamente durante todo el ciclo de vida de una aplicación. En lugar de considerar el desarrollo y la implementación como etapas independientes, DevOps propone integrarlas en un proceso continuo y coordinado.

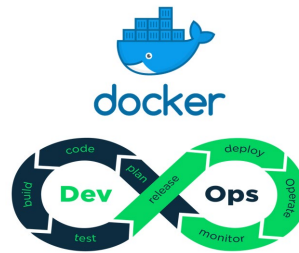
Dentro de esta filosofía aparecen una serie de principios fundamentales. La colaboración fomenta el trabajo conjunto entre todos los participantes del proyecto. La integración continua permite incorporar cambios al código de manera frecuente y controlada. La entrega continua busca que las aplicaciones se encuentren permanentemente preparadas para ser desplegadas. La automatización reduce la intervención manual en tareas repetitivas, disminuyendo errores y aumentando la eficiencia. El monitoreo permite supervisar constantemente el comportamiento de los sistemas, mientras que la mejora continua impulsa la optimización permanente de procesos, herramientas y metodologías de trabajo.

Docker surge precisamente dentro de este contexto. Su propósito es facilitar la construcción, distribución y ejecución de aplicaciones de forma consistente en distintos entornos, ayudando a resolver muchos de los problemas que históricamente existían entre los equipos de desarrollo y operaciones. A lo largo de la clase se analizará cómo Docker logra este objetivo y por qué se ha convertido en una de las herramientas más importantes del ecosistema DevOps moderno.

INTRODUCCIÓN A DOCKER

DevOps y Docker

- Integración y Entrega Continua (CI/CD)
- Automatización
- Colaboración
- Escalabilidad



Docker y su relación con DevOps

Una vez comprendidos los principios fundamentales de DevOps, resulta más sencillo entender por qué Docker se ha convertido en una de las herramientas más utilizadas dentro de este enfoque de trabajo. Docker fue diseñado para ayudar a resolver uno de los problemas más frecuentes en el desarrollo de software: las diferencias entre los distintos entornos donde una aplicación debe ejecutarse.

Durante muchos años fue habitual que una aplicación funcionara correctamente en la computadora de un desarrollador, pero presentara errores al ser trasladada a un entorno de pruebas o a un servidor de producción. Estas diferencias podían originarse por versiones distintas de bibliotecas, configuraciones incompatibles, dependencias faltantes o características específicas del sistema operativo. Como consecuencia, los equipos dedicaban una gran cantidad de tiempo a identificar y corregir problemas relacionados con el entorno de ejecución.

Docker aborda este inconveniente mediante el uso de contenedores. Un contenedor permite empaquetar una aplicación junto con todos los elementos necesarios para su funcionamiento, incluyendo bibliotecas, dependencias, configuraciones y herramientas requeridas. De esta manera, la aplicación puede ejecutarse de forma consistente en cualquier equipo o servidor que disponga de Docker, independientemente de las características particulares del entorno subyacente.

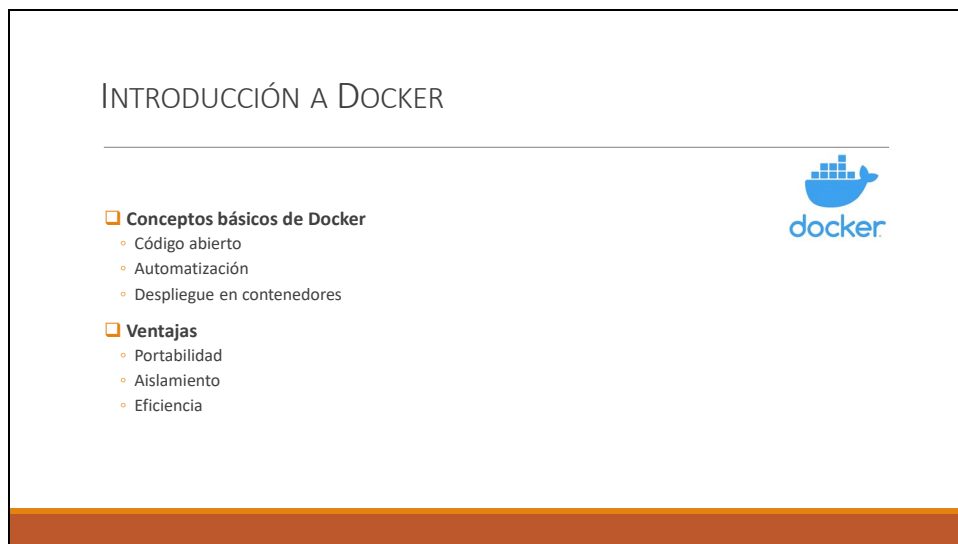
Esta característica resulta especialmente valiosa dentro de los procesos de Integración Continua y Entrega Continua, conocidos como CI/CD (Continuous Integration / Continuous Delivery). La Integración Continua consiste en incorporar de manera frecuente los cambios realizados por los desarrolladores en un repositorio común, permitiendo detectar errores de forma temprana. La Entrega Continua, por su parte, busca que las aplicaciones estén permanentemente preparadas para ser desplegadas en producción mediante procesos automatizados y controlados.

Docker facilita enormemente la implementación de estas prácticas porque permite crear entornos idénticos para desarrollo, pruebas y producción. Gracias a ello, los sistemas automatizados pueden construir aplicaciones, ejecutar pruebas, validar configuraciones y realizar despliegues utilizando exactamente las mismas condiciones en cada etapa del proceso.

Otro aspecto importante es la automatización. Mediante Docker es posible crear y eliminar entornos de ejecución en cuestión de segundos, sin necesidad de realizar configuraciones manuales complejas. Esto permite optimizar tareas como la ejecución de pruebas automatizadas, la implementación de nuevas versiones o la puesta en marcha de servicios temporales para distintos proyectos.

Además, Docker favorece la colaboración entre los equipos de trabajo. Al utilizar contenedores estandarizados, desarrolladores, administradores de sistemas y equipos de operaciones trabajan sobre entornos idénticos, reduciendo significativamente los problemas de compatibilidad y mejorando la comunicación entre las distintas áreas involucradas.

Finalmente, Docker contribuye a la escalabilidad de las aplicaciones. Al ser posible crear múltiples contenedores de manera rápida y eficiente, resulta sencillo aumentar la capacidad de procesamiento de un sistema cuando la demanda crece. Esta capacidad es fundamental en las arquitecturas modernas basadas en microservicios y en los entornos de computación en la nube, donde la flexibilidad y la adaptación dinámica de recursos constituyen requisitos esenciales.



Conceptos básicos de Docker

Docker es una plataforma de código abierto diseñada para facilitar el desarrollo, distribución y ejecución de aplicaciones mediante el uso de contenedores. Su principal objetivo es garantizar que una aplicación pueda ejecutarse de forma consistente en diferentes entornos, independientemente de las características particulares del sistema donde se despliegue.

El concepto fundamental de Docker es la contenedorización. Este enfoque consiste en encapsular una aplicación junto con todos los componentes necesarios para su funcionamiento dentro de una unidad denominada contenedor. De esta manera, la aplicación no depende de configuraciones externas específicas, ya que todo lo que necesita para ejecutarse se encuentra incluido dentro de dicho contenedor.

Un contenedor puede entenderse como un entorno aislado que incluye la aplicación, sus bibliotecas, dependencias, archivos de configuración y demás recursos requeridos para su ejecución. Gracias a este aislamiento, las aplicaciones pueden ejecutarse de manera predecible y uniforme, minimizando problemas relacionados con diferencias entre entornos de desarrollo, prueba y producción.

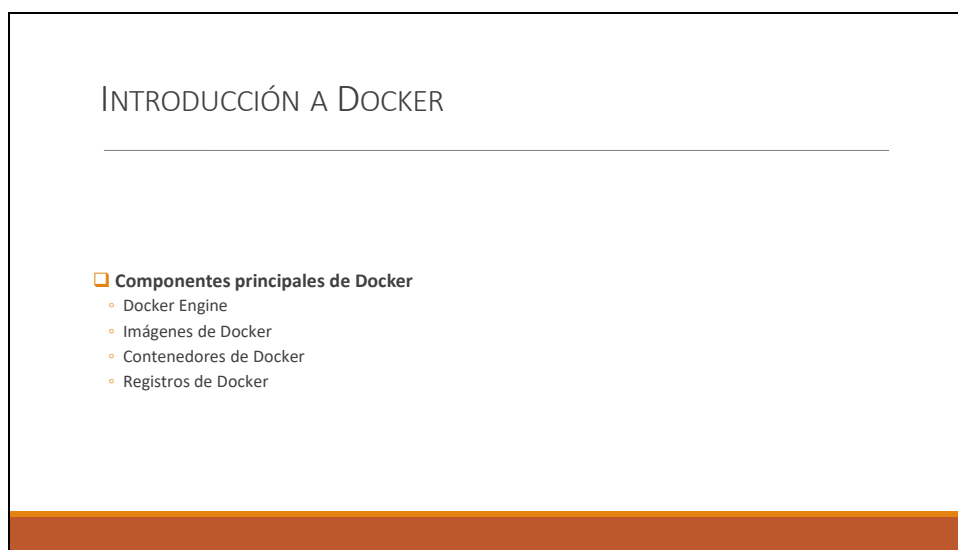
Una de las principales ventajas de Docker es la portabilidad. Un contenedor puede trasladarse fácilmente entre distintos equipos, servidores o proveedores de nube, manteniendo siempre el mismo comportamiento. Mientras el sistema anfitrión tenga Docker instalado, el contenedor podrá ejecutarse sin necesidad de realizar configuraciones adicionales complejas.

Otra característica importante es el aislamiento. Cada contenedor funciona de manera independiente respecto de los demás, disponiendo de sus propios procesos, configuraciones y recursos asignados. Esto

permite que múltiples aplicaciones convivan en un mismo servidor sin interferir entre sí, evitando conflictos de dependencias o incompatibilidades de versiones.

Finalmente, Docker se destaca por su eficiencia en el uso de recursos. A diferencia de las máquinas virtuales tradicionales, los contenedores no requieren ejecutar un sistema operativo completo para cada instancia. En su lugar, comparten el kernel del sistema operativo anfitrión, reduciendo significativamente el consumo de memoria, almacenamiento y capacidad de procesamiento. Esta característica permite ejecutar una gran cantidad de contenedores en un mismo servidor, aprovechando mejor la infraestructura disponible.

Gracias a estas ventajas, Docker se ha convertido en una herramienta fundamental dentro de los entornos modernos de desarrollo y operación de software, siendo ampliamente utilizada en arquitecturas basadas en microservicios, automatización de despliegues y plataformas de computación en la nube.



Componentes principales de Docker

Docker está formado por un conjunto de componentes que trabajan de manera coordinada para permitir la creación, distribución y ejecución de aplicaciones mediante contenedores. Comprender estos elementos resulta fundamental para entender el funcionamiento interno de la plataforma y el papel que cumple cada uno dentro del ecosistema Docker.

El componente central es **Docker Engine**, considerado el núcleo de la plataforma. Docker Engine es el software responsable de gestionar todos los recursos relacionados con contenedores, imágenes, redes y almacenamiento. Este componente permite que Docker funcione sobre distintos sistemas operativos y proporciona las herramientas necesarias para administrar el ciclo de vida completo de las aplicaciones contenedorizadas.

Dentro de Docker Engine se encuentran tres elementos principales. El primero es el **Docker Daemon** (*dockerd*), un servicio que se ejecuta en segundo plano y que se encarga de crear, iniciar, detener y administrar contenedores, imágenes, volúmenes y redes. Es el componente que realiza efectivamente las operaciones solicitadas por los usuarios o por otras aplicaciones.

El segundo elemento es la **API REST de Docker**, que proporciona una interfaz de comunicación estandarizada. Gracias a esta API, herramientas externas, aplicaciones de automatización y plataformas de orquestación pueden interactuar con Docker de forma programática, enviando solicitudes para administrar contenedores y recursos sin necesidad de utilizar comandos manuales.

El tercer componente es la **CLI (Command Line Interface)** o interfaz de línea de comandos. Se trata de la herramienta con la que los usuarios interactúan habitualmente mediante la terminal. Comandos como `docker run`, `docker build`, `docker ps` o `docker stop` son ejemplos de operaciones ejecutadas desde la CLI, que posteriormente son procesadas por Docker Engine.

Otro elemento fundamental son las **imágenes Docker**. Una imagen es una plantilla inmutable que contiene todo lo necesario para ejecutar una aplicación: código fuente, bibliotecas, dependencias, configuraciones y herramientas requeridas. Las imágenes sirven como punto de partida para la creación de contenedores y garantizan que todas las instancias generadas a partir de ellas compartan exactamente la misma configuración.

A partir de las imágenes se crean los **contenedores Docker**. Un contenedor puede entenderse como una instancia en ejecución de una imagen. Mientras que la imagen representa una plantilla estática, el contenedor constituye el entorno activo donde la aplicación se ejecuta, procesa información y realiza sus tareas. Es posible crear múltiples contenedores a partir de una misma imagen, permitiendo escalar aplicaciones de manera sencilla y eficiente.

Docker también incorpora los denominados **registros de imágenes** (*Docker Registries*), que funcionan como repositorios especializados para almacenar y distribuir imágenes. El registro más conocido es Docker Hub, aunque existen alternativas privadas y empresariales. Estos repositorios permiten compartir imágenes entre desarrolladores, equipos de trabajo y servidores, simplificando enormemente los procesos de despliegue.

Además de la gestión de imágenes y contenedores, Docker proporciona distintos mecanismos de conectividad mediante sus **modelos de red**. El modo **Bridge** es la configuración predeterminada y permite la comunicación entre contenedores dentro de una red virtual aislada. El modo **Host** elimina la capa de virtualización de red y permite que el contenedor utilice directamente la red del sistema anfitrión. Por último, el modo **None** crea contenedores sin conectividad de red, lo que puede resultar útil en escenarios donde se requiere un aislamiento total.

Todos estos componentes forman parte del ecosistema Docker y trabajan de manera integrada para proporcionar una plataforma flexible, portable y eficiente para la ejecución de aplicaciones modernas. Conocer la función de cada uno de ellos permite comprender mejor cómo Docker logra simplificar la distribución y administración de software en distintos entornos.

INTRODUCCIÓN A DOCKER

■ Imágenes de Docker

- Capas de la Imagen
- Dockerfile
- Registro de Imágenes
- Identificación y etiquetas

Imágenes de Docker

Las imágenes constituyen uno de los conceptos fundamentales dentro de Docker y representan la base sobre la cual se construyen los contenedores. Una imagen puede entenderse como una plantilla preconfigurada que contiene todos los elementos necesarios para crear y ejecutar un entorno de trabajo de manera consistente y reproducible.

Dentro de una imagen se incluyen los componentes requeridos para el funcionamiento de una aplicación, tales como el sistema operativo base, las bibliotecas necesarias, las dependencias de software, los archivos de configuración y la propia aplicación. Gracias a esta característica, cualquier contenedor creado a partir de una imagen dispondrá exactamente del mismo entorno de ejecución, independientemente del equipo o servidor donde se despliegue.

Una característica importante de las imágenes Docker es que están construidas mediante un sistema de capas. Cada instrucción utilizada durante el proceso de construcción genera una nueva capa que se agrega sobre las anteriores. Este mecanismo permite reutilizar componentes comunes entre diferentes imágenes, optimizando el uso del almacenamiento y acelerando los procesos de descarga, construcción y despliegue. Si varias imágenes comparten determinadas capas, Docker almacena una única copia de dichas capas y las reutiliza cuando sea necesario.

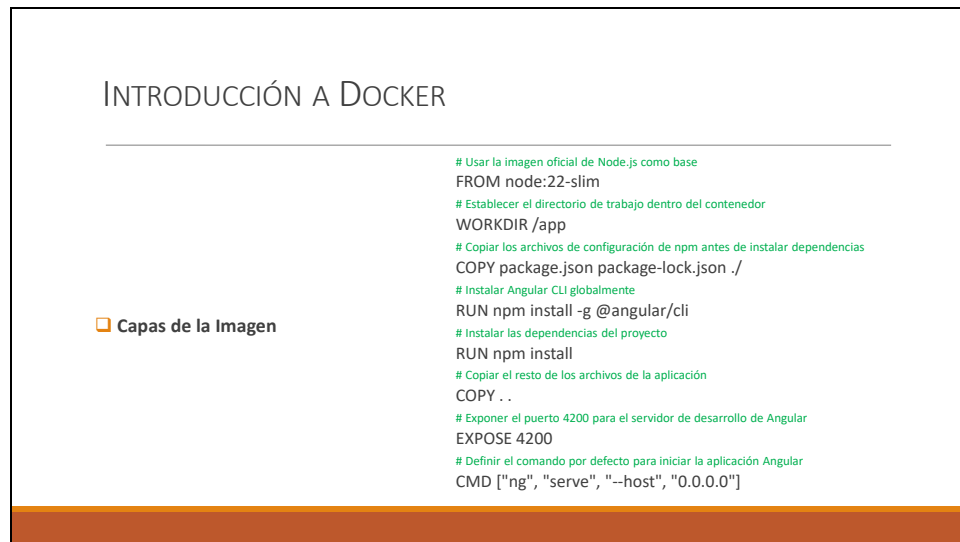
Para definir cómo debe construirse una imagen se utiliza un archivo denominado **Dockerfile**. Se trata de un archivo de texto que contiene una secuencia de instrucciones que indican paso a paso cómo generar la imagen. En este archivo es posible especificar la imagen base, copiar archivos, instalar software, configurar variables de entorno y definir el comportamiento que tendrá la aplicación al ejecutarse. El Dockerfile permite automatizar completamente la creación de imágenes y garantiza que el proceso pueda reproducirse de manera idéntica en cualquier entorno.

Las imágenes pueden almacenarse y distribuirse mediante los denominados **registros de imágenes** (*Image Registries*). Estos servicios funcionan como repositorios especializados donde los usuarios pueden publicar, compartir y descargar imágenes. El registro más conocido es Docker Hub, que ofrece miles de imágenes oficiales y mantenidas por la comunidad. Gracias a estos registros, los desarrolladores pueden reutilizar entornos ya preparados y reducir significativamente el tiempo necesario para implementar aplicaciones.

Por último, Docker utiliza un sistema de **identificación y etiquetado** mediante etiquetas o *tags*. Las etiquetas permiten diferenciar distintas versiones de una misma imagen y facilitan la gestión de actualizaciones. Por ejemplo, una imagen puede disponer de etiquetas como latest, 1.0, 2.0 o produccion, indicando distintas versiones o estados de desarrollo. Este mecanismo resulta fundamental

para controlar qué versión de una aplicación se está utilizando en cada entorno y para garantizar la trazabilidad de los despliegues.

En conjunto, las imágenes constituyen el elemento que proporciona portabilidad, consistencia y reproducibilidad dentro de Docker. Gracias a ellas es posible construir una vez una aplicación y ejecutarla de manera idéntica en cualquier entorno compatible, simplificando considerablemente los procesos de desarrollo, prueba y despliegue.



Capas de una Imagen Docker

En esta diapositiva se presenta un ejemplo real de un **Dockerfile**, el archivo utilizado para definir de manera automática cómo debe construirse una imagen Docker. Cada línea de este archivo representa una instrucción específica que Docker ejecuta durante el proceso de construcción, generando una nueva capa dentro de la imagen resultante.

La instrucción **FROM** es siempre el punto de partida de una imagen. Su función consiste en indicar cuál será la imagen base sobre la que se construirá el resto del entorno. Por ejemplo, es posible utilizar una imagen oficial de Ubuntu, Debian, Node.js o cualquier otra disponible en un registro de imágenes. Todas las instrucciones posteriores se aplicarán sobre esta base inicial.

La instrucción **WORKDIR** permite establecer el directorio de trabajo dentro del contenedor. A partir de ese momento, los comandos posteriores se ejecutarán utilizando dicha ubicación como referencia. Esto facilita la organización de archivos y simplifica la escritura de instrucciones posteriores dentro del Dockerfile.

Por su parte, la instrucción **COPY** se utiliza para copiar archivos y directorios desde el equipo anfitrión hacia la imagen que se está construyendo. Gracias a esta operación es posible incorporar el código fuente de una aplicación, archivos de configuración o cualquier otro recurso necesario para el funcionamiento del proyecto.

La instrucción **RUN** permite ejecutar comandos durante la construcción de la imagen. Es común utilizarla para instalar dependencias, actualizar paquetes, descargar herramientas o realizar configuraciones necesarias antes de que la imagen quede lista para ser utilizada. Cada comando RUN genera una nueva capa dentro de la imagen.

Otra instrucción frecuente es **EXPOSE**, que sirve para documentar los puertos que la aplicación utilizará dentro del contenedor. Aunque esta instrucción no publica automáticamente el puerto hacia el exterior,

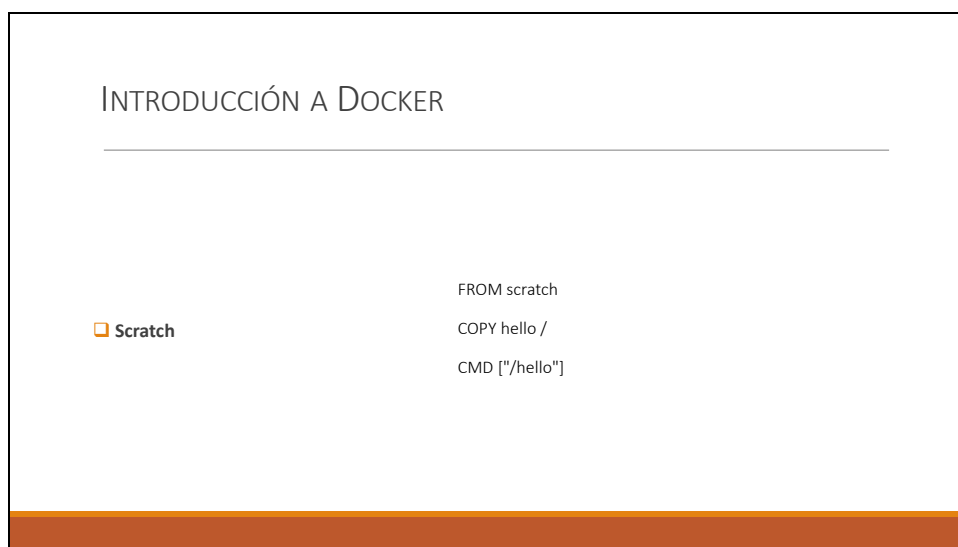
proporciona información útil sobre los servicios disponibles y facilita la administración de los contenedores.

Finalmente, la instrucción **CMD** define el comando principal que se ejecutará cuando el contenedor sea iniciado. Este comando representa normalmente el proceso principal de la aplicación, como un servidor web, una API o cualquier otro servicio que deba permanecer activo mientras el contenedor se encuentre en ejecución.

Uno de los aspectos más importantes de Docker es que cada una de estas instrucciones genera una **capa independiente**. Las capas se almacenan de manera separada y pueden reutilizarse entre distintas imágenes. Este mecanismo permite optimizar significativamente el almacenamiento y acelerar los procesos de construcción. Si una capa ya existe y no ha sufrido modificaciones, Docker puede reutilizarla directamente desde su caché sin necesidad de volver a ejecutarla.

Las capas poseen además una característica fundamental: son **inmutables**. Una vez creadas, no pueden modificarse. Cuando se realiza un cambio en una instrucción del Dockerfile, Docker genera una nueva capa que reemplaza a la anterior en la construcción de la imagen. Este enfoque garantiza consistencia, trazabilidad y reproducibilidad en todos los entornos donde se utilice la imagen.

Cuando finalmente se crea y ejecuta un contenedor a partir de una imagen, Docker agrega una capa adicional de escritura sobre todas las capas de solo lectura que conforman la imagen original. Esta capa temporal es donde se almacenan los cambios realizados durante la ejecución del contenedor, permitiendo que la imagen base permanezca intacta y pueda reutilizarse posteriormente sin alteraciones.



Scratch: una imagen vacía

Dentro del ecosistema Docker existe una imagen especial denominada **Scratch**, utilizada como punto de partida para construir imágenes extremadamente pequeñas y optimizadas. A diferencia de otras imágenes base, Scratch no contiene absolutamente ningún componente preinstalado. No incluye sistema operativo, bibliotecas, herramientas de administración ni programas adicionales. Puede considerarse como un entorno completamente vacío sobre el cual se construye una imagen desde cero.

Cuando en un Dockerfile se utiliza la instrucción `FROM scratch`, se está indicando a Docker que no debe utilizar ninguna imagen base existente y que la construcción comenzará desde un entorno vacío. Esto significa que todos los archivos necesarios para la ejecución de la aplicación deberán incorporarse explícitamente durante el proceso de construcción.

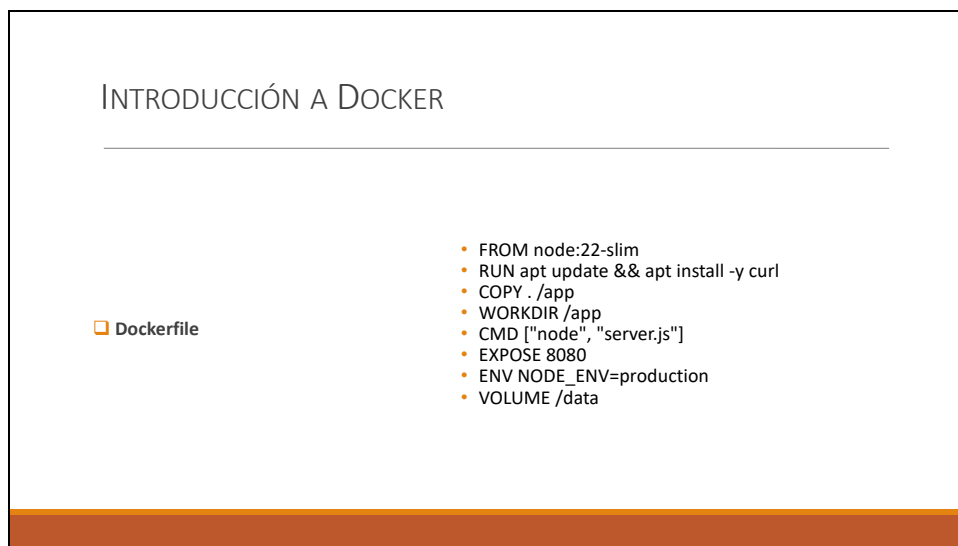
El uso de Scratch resulta especialmente útil cuando se dispone de aplicaciones compiladas que contienen internamente todas las dependencias requeridas para funcionar. En estos casos, basta con copiar el ejecutable dentro de la imagen y definirlo como proceso principal del contenedor. De esta manera, no es necesario instalar sistemas operativos, intérpretes, bibliotecas o herramientas adicionales que aumentarían innecesariamente el tamaño de la imagen.

Una de las principales ventajas de este enfoque es la reducción drástica del tamaño final de la imagen. Al incluir únicamente los archivos estrictamente necesarios para ejecutar la aplicación, es posible generar contenedores extremadamente livianos. Esto se traduce en descargas más rápidas, menor consumo de almacenamiento y tiempos de despliegue más reducidos.

Otra ventaja importante es la disminución de la superficie de ataque. Al no existir herramientas adicionales, paquetes innecesarios ni servicios auxiliares dentro del contenedor, se reduce la cantidad de componentes potencialmente vulnerables que podrían ser explotados por un atacante. Esta característica convierte a Scratch en una opción muy atractiva para aplicaciones donde la seguridad constituye un requisito prioritario.

Asimismo, las imágenes basadas en Scratch suelen consumir menos recursos y simplifican la administración de dependencias, ya que únicamente contienen los elementos indispensables para la ejecución de la aplicación. Por este motivo, son ampliamente utilizadas en entornos modernos donde se busca maximizar la eficiencia y minimizar la complejidad de los contenedores.

En resumen, Scratch representa el nivel más básico y minimalista dentro de Docker. Su utilización permite construir imágenes extremadamente pequeñas, seguras y eficientes, siendo una herramienta fundamental para la creación de contenedores optimizados en entornos de producción.



Dockerfile

El **Dockerfile** es uno de los componentes más importantes de Docker, ya que permite definir de manera automatizada el proceso de construcción de una imagen. Se trata de un archivo de texto que contiene una serie de instrucciones organizadas secuencialmente, las cuales indican a Docker cómo debe crearse el entorno que posteriormente dará origen a uno o más contenedores.

Puede pensarse al Dockerfile como una receta de construcción. Así como una receta describe paso a paso los ingredientes y procedimientos necesarios para preparar un plato, el Dockerfile describe todas las acciones necesarias para construir una imagen de manera reproducible y consistente. Gracias a este

enfoque, cualquier persona o sistema puede generar exactamente la misma imagen utilizando el mismo archivo.

La instrucción **FROM** establece la imagen base sobre la cual se construirá el resto del entorno. Esta imagen inicial proporciona el punto de partida y puede corresponder a una distribución Linux, un entorno de ejecución específico o una imagen previamente preparada por terceros.

La instrucción **RUN** permite ejecutar comandos durante el proceso de construcción de la imagen. Habitualmente se utiliza para instalar paquetes, actualizar software, crear directorios o realizar configuraciones necesarias para que la aplicación funcione correctamente.

La instrucción **COPY** se utiliza para copiar archivos y directorios desde el sistema anfitrión hacia la imagen en construcción. Esto permite incorporar el código fuente de la aplicación, archivos de configuración, recursos estáticos y cualquier otro elemento necesario para el funcionamiento del proyecto.

Por su parte, **WORKDIR** define el directorio de trabajo que utilizarán las instrucciones posteriores. Una vez establecido, los comandos que se ejecuten dentro del Dockerfile tomarán ese directorio como referencia, facilitando la organización de archivos y simplificando la escritura de rutas.

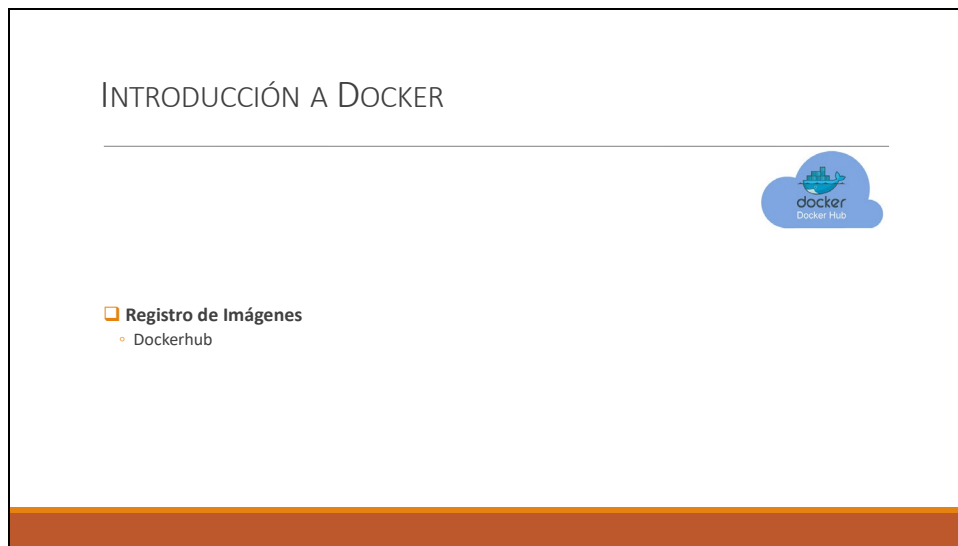
La instrucción **CMD** especifica el comando principal que se ejecutará cuando el contenedor sea iniciado. Generalmente corresponde al proceso principal de la aplicación, como un servidor web, una API o cualquier servicio que deba permanecer activo durante la vida útil del contenedor.

La instrucción **EXPOSE** permite documentar los puertos que la aplicación utiliza dentro del contenedor. Aunque no realiza la publicación automática de dichos puertos, proporciona información importante para quienes administran o despliegan la aplicación.

Docker también permite definir variables de entorno mediante la instrucción **ENV**. Estas variables pueden utilizarse para almacenar configuraciones, parámetros de ejecución o información necesaria para el funcionamiento de la aplicación, evitando modificar directamente el código fuente cuando cambian ciertos valores.

Otra instrucción relevante es **VOLUME**, utilizada para definir ubicaciones destinadas al almacenamiento persistente de datos. Los volúmenes permiten conservar información incluso cuando los contenedores son eliminados o recreados, separando los datos del ciclo de vida del contenedor.

Una de las mayores ventajas del Dockerfile es que permite automatizar completamente la construcción de imágenes. En lugar de realizar configuraciones manuales en cada servidor o equipo de trabajo, todas las acciones quedan documentadas dentro del archivo. Esto garantiza consistencia, facilita la colaboración entre equipos y permite recrear exactamente el mismo entorno tantas veces como sea necesario, independientemente del lugar donde se construya la imagen.



Registro de Imágenes

Las imágenes Docker no se almacenan únicamente en los equipos donde son creadas. Para facilitar su distribución y reutilización, Docker utiliza repositorios especializados denominados **registros de imágenes** (*Image Registries*). Estos servicios permiten almacenar, compartir y descargar imágenes de manera centralizada, simplificando enormemente los procesos de desarrollo y despliegue.

El registro más conocido y utilizado es [Docker Hub](#), considerado el repositorio público oficial de Docker. Su funcionamiento puede compararse con el de plataformas como GitHub, aunque en lugar de almacenar código fuente, almacena imágenes Docker listas para ser utilizadas. Esto permite que desarrolladores y organizaciones compartan entornos completos de forma rápida y estandarizada.

Docker Hub ofrece una gran cantidad de imágenes oficiales mantenidas por los propios desarrolladores de cada tecnología o por la comunidad. Entre ellas se encuentran distribuciones Linux como Ubuntu, servidores web como Apache y Nginx, motores de bases de datos como MySQL y PostgreSQL, así como entornos de desarrollo para lenguajes y plataformas como Node.js, Python, Java y muchas otras. Gracias a estas imágenes preconstruidas, es posible disponer de entornos funcionales en cuestión de minutos sin necesidad de realizar instalaciones manuales complejas.

Además de descargar imágenes existentes, los usuarios pueden publicar sus propias imágenes en Docker Hub. Esto resulta especialmente útil para compartir aplicaciones con otros desarrolladores, distribuir versiones de software dentro de un equipo de trabajo o implementar sistemas de despliegue automatizado. Una vez publicada una imagen, cualquier usuario autorizado puede descargarla y ejecutarla en su propio entorno.

En entornos empresariales también es habitual utilizar **registros privados**, que funcionan de manera similar a Docker Hub pero restringen el acceso únicamente a usuarios autorizados. Estos registros permiten almacenar imágenes internas de la organización, proteger información sensible y controlar el acceso a aplicaciones corporativas. De esta forma, las empresas pueden mantener un repositorio centralizado de imágenes que será utilizado por los distintos equipos de desarrollo, pruebas y producción.

La existencia de registros de imágenes constituye uno de los pilares fundamentales del ecosistema Docker, ya que permite distribuir aplicaciones de manera rápida, consistente y segura entre diferentes equipos, servidores y entornos de ejecución. Gracias a este mecanismo, una imagen puede construirse una única vez y luego ser utilizada tantas veces como sea necesario en cualquier sistema compatible con Docker.

INTRODUCCIÓN A DOCKER

Contenedores de Docker

- ¿Qué es un contenedor?
- Características principales
- ¿Cómo funcionan?



Contenedores de Docker

Los contenedores constituyen el elemento central de Docker y representan la unidad de ejecución de una aplicación. Mientras que una imagen puede considerarse una plantilla o modelo predefinido, un contenedor es una instancia activa creada a partir de dicha imagen. En otras palabras, la imagen define cómo debe ser el entorno, mientras que el contenedor corresponde al entorno funcionando efectivamente.

Cuando Docker inicia un contenedor, toma como base una imagen previamente construida y genera un entorno de ejecución aislado donde la aplicación puede ejecutarse. Este entorno contiene todos los archivos, dependencias, configuraciones y recursos definidos en la imagen, permitiendo que la aplicación funcione de manera consistente independientemente del sistema donde se ejecute.

Una de las principales características de los contenedores es su **aislamiento**. Cada contenedor ejecuta sus procesos de manera independiente respecto de otros contenedores y respecto del sistema anfitrión. Esto permite que múltiples aplicaciones convivan en un mismo servidor sin interferir entre sí, incluso cuando utilizan versiones diferentes de bibliotecas o configuraciones particulares.

Los contenedores también se destacan por su **portabilidad**. Un mismo contenedor puede ejecutarse en distintos equipos, servidores o plataformas de nube manteniendo exactamente el mismo comportamiento. Esta característica elimina gran parte de los problemas relacionados con diferencias entre entornos de desarrollo, prueba y producción.

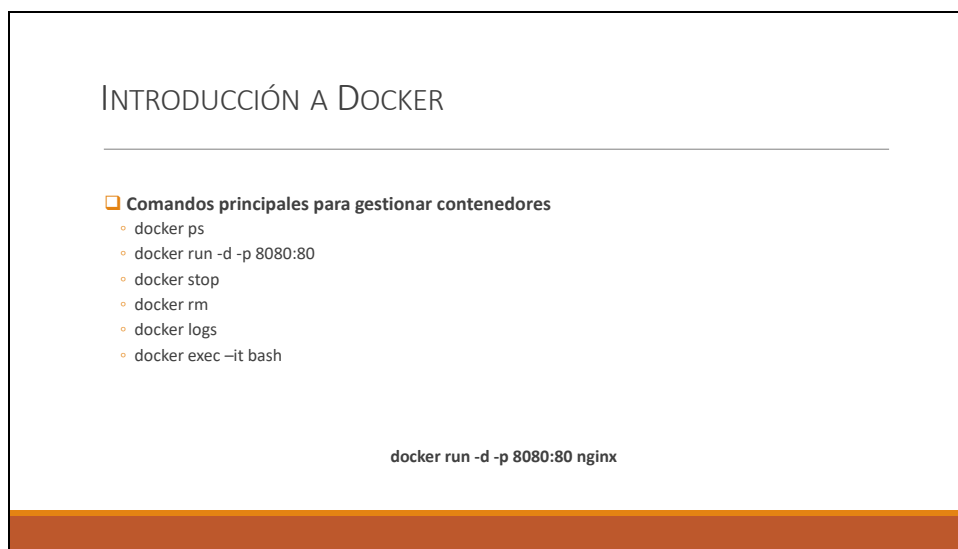
Otra ventaja importante es su **rapidez y ligereza**. Debido a que los contenedores comparten el kernel del sistema operativo anfitrión, requieren muchos menos recursos que una máquina virtual tradicional. Esto permite iniciarlos en cuestión de segundos y ejecutar una gran cantidad de ellos sobre una misma infraestructura física.

Durante su ciclo de vida, los contenedores pueden administrarse mediante distintas operaciones. Es posible iniciarlos, detenerlos, reiniciarlos, pausarlos, eliminarlos o recrearlos según las necesidades del sistema. Esta flexibilidad facilita enormemente las tareas de mantenimiento, actualización y despliegue de aplicaciones.

Además, Docker permite que los contenedores se comuniquen entre sí mediante redes virtuales administradas por la propia plataforma. Gracias a este mecanismo, diferentes componentes de una aplicación, como servidores web, bases de datos o servicios de autenticación, pueden intercambiar información de manera segura y organizada sin necesidad de configuraciones complejas.

Por último, es importante considerar la persistencia de datos. Por defecto, la información almacenada dentro de un contenedor se pierde cuando este es eliminado. Para resolver esta limitación, Docker incorpora el concepto de **volúmenes**, que permiten almacenar datos fuera del ciclo de vida del contenedor. De esta manera, la información puede conservarse aunque los contenedores sean destruidos, actualizados o reemplazados.

En conjunto, los contenedores proporcionan un entorno aislado, portable y eficiente para la ejecución de aplicaciones, convirtiéndose en la pieza fundamental sobre la cual se construyen las arquitecturas modernas basadas en Docker.



Comandos principales para gestionar contenedores

Una vez comprendidos los conceptos fundamentales de Docker, es posible comenzar a trabajar con contenedores reales mediante la interfaz de línea de comandos. Docker proporciona un conjunto de instrucciones que permiten crear, administrar y monitorear contenedores de forma sencilla y eficiente. Estos comandos constituyen la base de la administración diaria de aplicaciones contenedorizadas.

Uno de los comandos más importantes es **docker run**, utilizado para crear e iniciar contenedores a partir de una imagen. Si la imagen indicada no se encuentra almacenada localmente, Docker la descargará automáticamente desde el registro correspondiente antes de crear el contenedor. Este comando representa habitualmente el punto de partida para la ejecución de aplicaciones dentro de Docker.

Es común acompañar `docker run` con distintas opciones que modifican su comportamiento. La opción **-d** (*detached mode*) permite ejecutar el contenedor en segundo plano, liberando la terminal para continuar trabajando. Por su parte, la opción **-p** se utiliza para mapear puertos entre el contenedor y el sistema anfitrión, permitiendo que los servicios ejecutados dentro del contenedor puedan ser accedidos desde el exterior.

Por ejemplo, el comando:

```
docker run -d -p 8080:80 nginx
```

indica a Docker que debe crear un contenedor utilizando la imagen Nginx. Si dicha imagen no existe localmente, será descargada automáticamente. El contenedor se ejecutará en segundo plano y el puerto 80 interno del servicio web será publicado a través del puerto 8080 del sistema anfitrión. Como resultado, los usuarios podrán acceder al servidor web desde un navegador utilizando la dirección correspondiente al host y el puerto 8080.

Para visualizar los contenedores que se encuentran actualmente en ejecución se utiliza el comando **docker ps**. Este comando muestra información relevante como el identificador del contenedor, la imagen utilizada, el estado de ejecución, los puertos publicados y el nombre asignado al contenedor.

Cuando se requiere detener un contenedor en ejecución, se utiliza el comando **docker stop**, seguido del identificador o nombre del contenedor. Docker enviará una señal de detención controlada al proceso principal, permitiendo finalizar su ejecución de manera ordenada.

Una vez que un contenedor ya no resulta necesario, puede eliminarse mediante el comando **docker rm**. Esta operación libera los recursos asociados al contenedor y elimina su configuración. Es importante destacar que un contenedor debe estar detenido antes de poder ser eliminado.

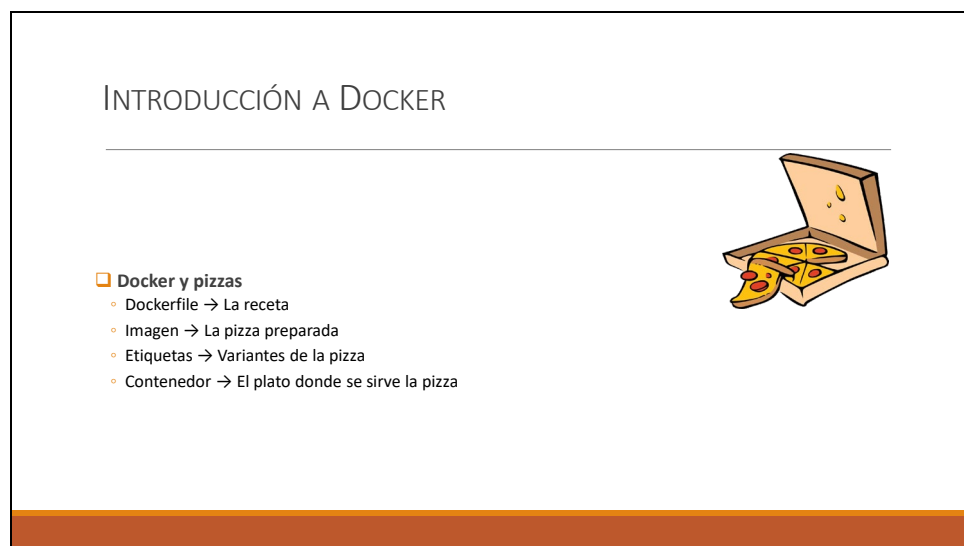
Para consultar la salida generada por una aplicación en ejecución se utiliza el comando **docker logs**. Este comando permite visualizar los registros producidos por el contenedor, resultando especialmente útil para tareas de monitoreo, diagnóstico y resolución de problemas.

Finalmente, Docker permite acceder directamente al interior de un contenedor mediante el comando **docker exec**. Una de las formas más habituales consiste en iniciar una terminal interactiva utilizando:

```
docker exec -it nombre_contenedor bash
```

Esta operación permite ejecutar comandos dentro del contenedor como si se estuviera trabajando directamente sobre el sistema operativo que contiene, facilitando tareas de inspección, configuración y mantenimiento.

El dominio de estos comandos básicos constituye el primer paso para administrar contenedores de manera efectiva y comprender el funcionamiento práctico de Docker en entornos reales de desarrollo y producción.



Esta analogía ayuda muchísimo a entender Docker.

El Dockerfile es la receta.

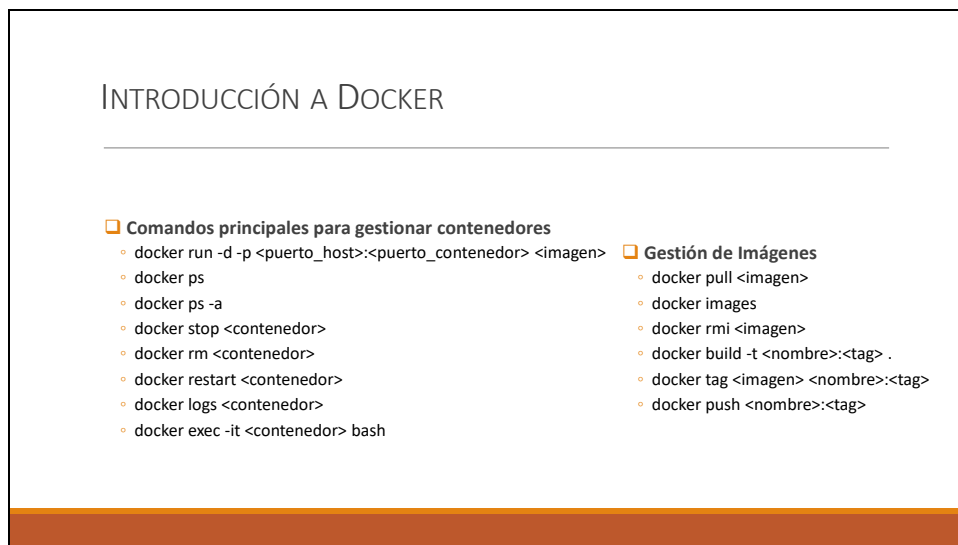
La imagen es la pizza ya preparada.

Las etiquetas son variantes de la pizza.

Y el contenedor es el plato donde se sirve.

La gran ventaja es que siempre obtenemos exactamente la misma pizza siguiendo la misma receta.

Eso es justamente lo que busca Docker:
reproducibilidad.



Docker y pizzas: una analogía para comprender los conceptos básicos

En esta analogía, el **Dockerfile** representa la receta de la pizza. Allí se encuentran detalladas todas las instrucciones necesarias para obtener el resultado final: los ingredientes que se utilizarán, los pasos de preparación y las acciones que deben realizarse durante el proceso. De manera similar, un Dockerfile contiene las instrucciones que Docker utilizará para construir una imagen.

La **imagen Docker** equivale a la pizza ya preparada siguiendo la receta. Se trata de un producto terminado que puede utilizarse tantas veces como sea necesario para generar nuevas instancias. Del mismo modo que una pizzería puede preparar varias pizzas idénticas utilizando la misma receta, Docker puede crear múltiples contenedores a partir de una misma imagen.

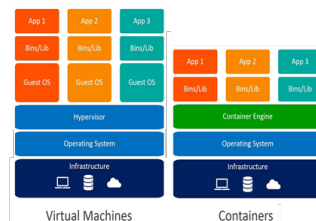
Las **etiquetas** o *tags* pueden compararse con las distintas variantes de una pizza. Por ejemplo, una misma pizzería puede ofrecer versiones diferentes de una receta base, como mozzarella, jamón y morrones o cuatro quesos. En Docker, las etiquetas permiten identificar distintas versiones de una imagen, facilitando el control de cambios y la gestión de actualizaciones.

Finalmente, el **contenedor** puede asociarse al plato donde se sirve la pizza para ser consumida. Mientras la imagen representa el producto preparado, el contenedor constituye la instancia activa y lista para utilizar. Es el entorno donde la aplicación se encuentra efectivamente ejecutándose y prestando servicio.

El objetivo de esta analogía es el concepto de **reproducibilidad**. Siempre que se utilice la misma receta y los mismos ingredientes, será posible obtener una pizza idéntica. Del mismo modo, cuando Docker construye una imagen a partir de un Dockerfile determinado, cualquier contenedor generado desde esa imagen tendrá exactamente las mismas características y comportamiento. Esta capacidad de reproducir entornos de forma consistente es una de las razones por las cuales Docker se ha convertido en una herramienta fundamental dentro del desarrollo y la operación de aplicaciones modernas.

INTRODUCCIÓN A DOCKER

- Diferencias entre Docker y VM
- Ventajas y desventajas



Diferencias entre Docker y las Máquinas Virtuales

Docker y las máquinas virtuales son tecnologías que persiguen un objetivo similar: permitir la ejecución de aplicaciones en entornos aislados e independientes. Sin embargo, la forma en que logran ese aislamiento es muy diferente, y esa diferencia tiene un impacto directo sobre el consumo de recursos, el rendimiento y la flexibilidad de la infraestructura.

Las **máquinas virtuales (Virtual Machines o VM)** se basan en la virtualización de hardware. Para ello utilizan un componente denominado **hypervisor**, que actúa como intermediario entre el hardware físico y las máquinas virtuales. Su función consiste en administrar los recursos disponibles del servidor, como procesador, memoria, almacenamiento y red, distribuyéndolos entre múltiples sistemas virtuales independientes. Entre los hypervisores más utilizados se encuentran VirtualBox, VMware, Hyper-V y KVM.

Cada máquina virtual funciona como una computadora completa. Esto significa que posee su propio sistema operativo, su propio kernel, sus controladores, servicios y aplicaciones. Gracias a esta arquitectura, es posible ejecutar simultáneamente distintos sistemas operativos sobre un mismo equipo físico, por ejemplo Windows Server, Ubuntu y Debian, cada uno completamente aislado de los demás.

Esta independencia proporciona un alto nivel de aislamiento y compatibilidad, pero también implica un costo importante en términos de recursos. Cada máquina virtual requiere memoria RAM propia, espacio de almacenamiento para el sistema operativo completo, procesos de administración y tiempo de arranque considerable. Como consecuencia, el consumo de recursos suele ser elevado y la cantidad de máquinas virtuales que pueden ejecutarse simultáneamente resulta limitada por la capacidad del hardware disponible.

Docker adopta un enfoque diferente. En lugar de virtualizar computadoras completas, Docker se enfoca en la virtualización de aplicaciones y procesos. Para ello utiliza un componente denominado **Docker Engine**, también conocido como **Container Engine**, que administra la creación y ejecución de contenedores.

A diferencia de las máquinas virtuales, los contenedores no incorporan un sistema operativo completo propio. Todos los contenedores comparten el kernel del sistema operativo anfitrión y únicamente contienen la aplicación, sus dependencias, bibliotecas y configuraciones necesarias para funcionar. Gracias a este diseño, los contenedores son significativamente más livianos que las máquinas virtuales.

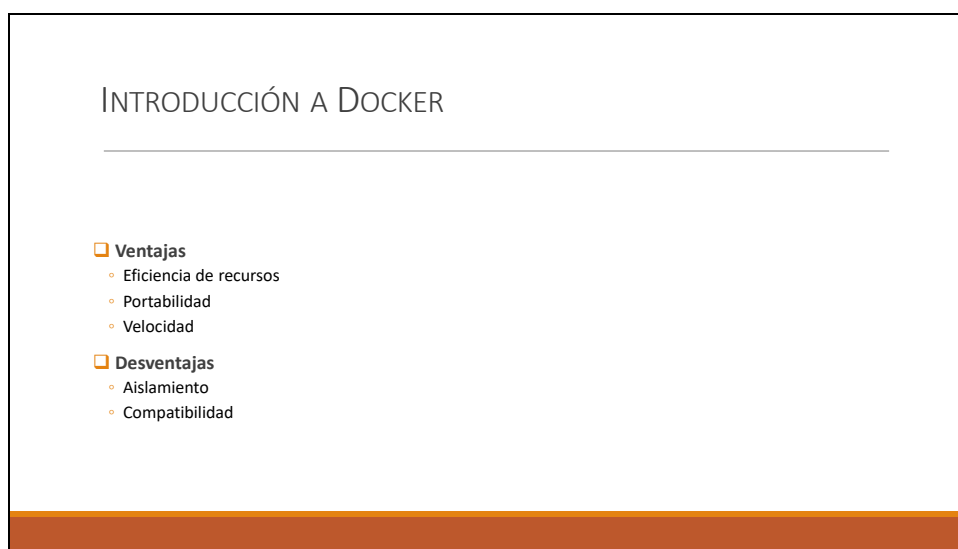
Esta arquitectura permite iniciar contenedores en cuestión de segundos o incluso milisegundos, reducir considerablemente el consumo de memoria y almacenamiento, y ejecutar una cantidad mucho mayor de instancias sobre el mismo hardware. Por esta razón, Docker se ha convertido en una herramienta fundamental para arquitecturas modernas basadas en microservicios y despliegues escalables.

Otro concepto importante dentro de Docker es la diferencia entre **imagen** y **contenedor**. La imagen actúa como una plantilla que contiene todo lo necesario para ejecutar una aplicación, mientras que el contenedor representa una instancia activa creada a partir de esa imagen. De esta manera, una misma imagen puede utilizarse para generar múltiples contenedores idénticos.

A pesar de sus ventajas, Docker presenta ciertas limitaciones en comparación con las máquinas virtuales. El principal aspecto es el nivel de aislamiento. Como todos los contenedores comparten el mismo kernel del sistema anfitrión, el aislamiento no es tan absoluto como el que proporciona una máquina virtual completa. En entornos donde se requiere una separación extremadamente estricta entre aplicaciones o usuarios, las máquinas virtuales continúan ofreciendo ventajas importantes.

Por este motivo, en muchas organizaciones ambas tecnologías se utilizan de manera complementaria. Es habitual que la infraestructura se construya sobre máquinas virtuales y que, dentro de ellas, se ejecuten múltiples contenedores Docker. Este enfoque combina el alto nivel de aislamiento proporcionado por la virtualización tradicional con la eficiencia, portabilidad y escalabilidad que ofrecen los contenedores.

En la actualidad, la combinación de máquinas virtuales y contenedores constituye una práctica estándar en entornos de computación en la nube, centros de datos corporativos y plataformas modernas de desarrollo y despliegue de aplicaciones.



Ventajas y desventajas de Docker

Docker se ha convertido en una de las tecnologías más utilizadas para el desarrollo y despliegue de aplicaciones debido a las numerosas ventajas que ofrece frente a los enfoques tradicionales. Su diseño basado en contenedores permite optimizar recursos, simplificar la distribución de software y acelerar los procesos de implementación, características que han impulsado su adopción en organizaciones de todos los tamaños.

Una de sus principales ventajas es la **eficiencia en el uso de recursos**. Como los contenedores comparten el kernel del sistema operativo anfitrión, no es necesario ejecutar un sistema operativo completo para cada instancia. Esto reduce significativamente el consumo de memoria, almacenamiento y capacidad de procesamiento en comparación con las máquinas virtuales tradicionales.

Otra ventaja importante es la **portabilidad**. Un contenedor Docker puede ejecutarse de la misma manera en distintos equipos, servidores o plataformas de nube, siempre que dispongan de Docker instalado. Esto elimina gran parte de los problemas relacionados con diferencias de configuración entre entornos de desarrollo, prueba y producción, garantizando un comportamiento consistente de las aplicaciones.

Docker también se destaca por su **velocidad**. Los contenedores pueden iniciarse y detenerse en cuestión de segundos, ya que no requieren el arranque completo de un sistema operativo. Esta característica facilita enormemente la creación de entornos temporales, la ejecución de pruebas automatizadas y el despliegue rápido de nuevas versiones de software.

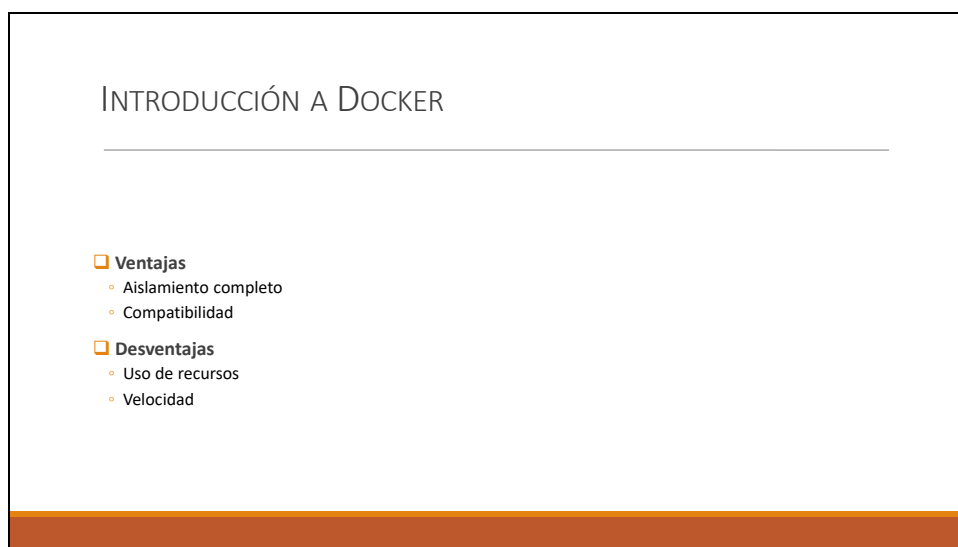
Además, Docker favorece la **consistencia de los despliegues**. Al empaquetar la aplicación junto con todas sus dependencias y configuraciones, se reduce la posibilidad de errores causados por diferencias entre entornos. Esto permite construir una aplicación una sola vez y ejecutarla de manera idéntica en cualquier infraestructura compatible.

Otra fortaleza importante es la facilidad para implementar procesos de **automatización y escalabilidad**. Los contenedores pueden crearse, replicarse o eliminarse rápidamente según la demanda del sistema, lo que los convierte en una solución ideal para arquitecturas modernas basadas en microservicios y plataformas de computación en la nube.

Sin embargo, Docker también presenta algunas limitaciones que deben ser consideradas. La principal está relacionada con el **aislamiento**. Aunque los contenedores se ejecutan de forma independiente, comparten el kernel del sistema operativo anfitrión. Por este motivo, el nivel de aislamiento no es tan estricto como el proporcionado por una máquina virtual completa, donde cada instancia dispone de su propio sistema operativo.

Otra desventaja es la **dependencia del sistema anfitrión**. Debido a que los contenedores comparten el kernel del host, existen ciertas restricciones de compatibilidad. Por ejemplo, un contenedor diseñado para ejecutarse sobre Linux depende de características propias de dicho kernel, lo que requiere mecanismos adicionales cuando se trabaja sobre otros sistemas operativos.

En conclusión, Docker ofrece importantes beneficios en términos de eficiencia, portabilidad, velocidad y escalabilidad, convirtiéndose en una herramienta fundamental dentro de los entornos modernos de desarrollo y operaciones. No obstante, es importante comprender sus limitaciones para determinar cuándo resulta conveniente utilizar contenedores, máquinas virtuales o una combinación de ambas tecnologías según las necesidades específicas de cada proyecto.



Ventajas y desventajas de las Máquinas Virtuales

Las máquinas virtuales continúan siendo una tecnología ampliamente utilizada en entornos empresariales debido a las ventajas que ofrecen en materia de aislamiento, compatibilidad y seguridad.

Aunque los contenedores han ganado una gran popularidad en los últimos años, las máquinas virtuales siguen desempeñando un papel fundamental dentro de muchas infraestructuras modernas.

Una de sus principales ventajas es el **aislamiento completo** entre entornos. Cada máquina virtual posee su propio sistema operativo, su propio kernel y sus propios recursos asignados. Esto significa que los procesos que se ejecutan dentro de una máquina virtual se encuentran completamente separados de los de las demás, proporcionando un nivel de aislamiento superior al de los contenedores. Esta característica resulta especialmente valiosa en entornos donde la seguridad y la separación de cargas de trabajo son requisitos críticos.

Otra ventaja importante es la **compatibilidad**. Como cada máquina virtual incorpora un sistema operativo completo, es posible ejecutar diferentes sistemas operativos sobre un mismo servidor físico. Por ejemplo, una organización puede ejecutar simultáneamente máquinas virtuales con Windows Server, Ubuntu, Debian o cualquier otra plataforma compatible con el hypervisor utilizado. Esta flexibilidad permite adaptar la infraestructura a las necesidades específicas de cada aplicación o servicio.

Sin embargo, estas ventajas tienen un costo asociado. Una de las principales desventajas de las máquinas virtuales es el **mayor consumo de recursos**. Cada instancia requiere memoria RAM propia, espacio de almacenamiento para el sistema operativo completo, recursos de procesamiento y servicios adicionales necesarios para su funcionamiento. Como consecuencia, la cantidad de máquinas virtuales que pueden ejecutarse simultáneamente suele ser menor que la cantidad de contenedores que podrían desplegarse sobre el mismo hardware.

Otra limitación importante es la **velocidad de inicio**. Debido a que una máquina virtual debe arrancar un sistema operativo completo antes de que la aplicación pueda comenzar a funcionar, los tiempos de inicio suelen medirse en segundos o incluso minutos. En contraste, los contenedores pueden iniciarse casi instantáneamente, ya que únicamente ejecutan los procesos necesarios para la aplicación.

Por este motivo, en la actualidad muchas organizaciones adoptan un enfoque híbrido que combina ambas tecnologías. Es habitual utilizar máquinas virtuales como base de la infraestructura, aprovechando su alto nivel de aislamiento y compatibilidad, mientras que dentro de ellas se ejecutan contenedores Docker para obtener mayor eficiencia, portabilidad y facilidad de despliegue. Esta combinación permite aprovechar las fortalezas de cada tecnología y constituye una práctica ampliamente utilizada en entornos de computación en la nube y centros de datos modernos.

INTRODUCCIÓN A DOCKER

Instalación de Docker

- `sudo apt-get remove docker docker-engine docker.io containerd runc`
- `sudo apt-get update`
- `sudo apt-get upgrade -y`
- `sudo apt-get install -y ca-certificates curl gnupg lsb-release`
- `sudo mkdir -p /etc/apt/keyrings`
- `curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`
- `echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \n https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null`
- `sudo apt-get update`
- `sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`
- `sudo docker run hello-world`
- `sudo usermod -aG docker $USER`
- `newgrp docker`

Instalación de Docker en Ubuntu

Docker puede instalarse en Ubuntu utilizando los repositorios oficiales proporcionados por el proyecto. Aunque existen distintos métodos de instalación, la utilización del repositorio oficial garantiza el acceso a versiones actualizadas, seguras y mantenidas directamente por los desarrolladores de Docker.

El primer paso consiste en eliminar posibles versiones antiguas o paquetes obsoletos relacionados con Docker que pudieran encontrarse instalados en el sistema. Esta limpieza previa evita conflictos durante la instalación y asegura que todos los componentes se instalen correctamente desde las fuentes oficiales.

A continuación, se actualiza la información de los repositorios del sistema mediante las herramientas de gestión de paquetes de Ubuntu. Este procedimiento permite que el sistema operativo conozca las versiones más recientes de los paquetes disponibles antes de iniciar la instalación.

Posteriormente, se instalan una serie de herramientas necesarias para agregar repositorios externos de forma segura. Entre ellas se incluyen utilidades para la gestión de certificados digitales, descarga de archivos mediante HTTPS y administración de claves criptográficas utilizadas para verificar la autenticidad de los paquetes.

El siguiente paso consiste en descargar e instalar la **clave GPG oficial de Docker**. Esta clave permite verificar la integridad y autenticidad de los paquetes descargados desde el repositorio oficial, garantizando que el software proviene efectivamente del proyecto Docker y no ha sido alterado durante la distribución.

Una vez incorporada la clave de seguridad, se agrega el repositorio oficial de Docker a la configuración del sistema. A partir de ese momento, Ubuntu podrá descargar e instalar directamente las versiones oficiales de Docker utilizando el gestor de paquetes estándar.

Con el repositorio configurado, se procede a instalar los componentes principales de la plataforma, incluyendo **Docker Engine**, responsable de la gestión de contenedores, y **Docker Compose**, una herramienta que permite definir y administrar aplicaciones compuestas por múltiples contenedores mediante archivos de configuración declarativos.

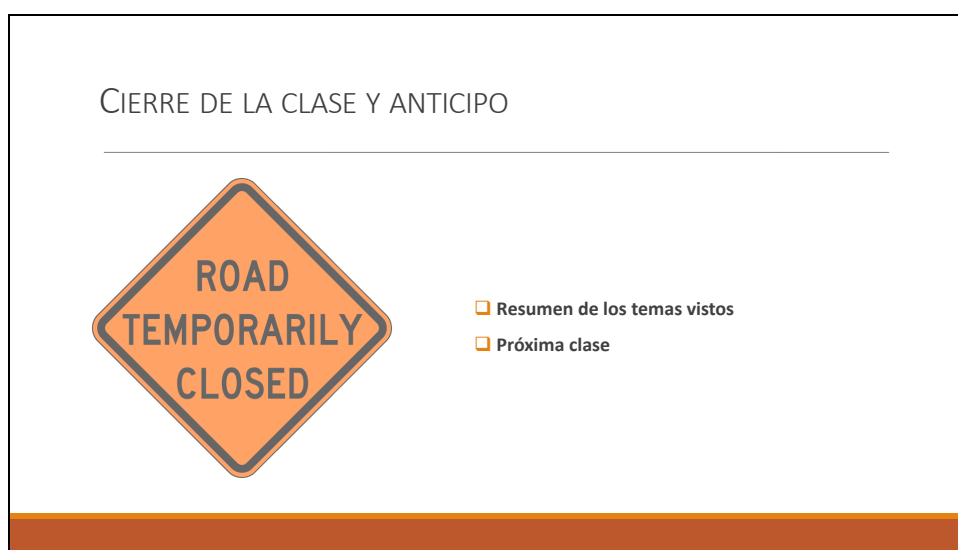
Finalizada la instalación, es recomendable realizar una prueba básica para verificar que Docker funciona correctamente. Para ello se ejecuta el comando:

```
docker run hello-world
```

Este comando descarga una imagen de prueba oficial, crea un contenedor y ejecuta una serie de verificaciones internas. Si todo funciona correctamente, Docker mostrará un mensaje indicando que la instalación se ha realizado con éxito y que el motor de contenedores se encuentra operativo.

Por último, suele realizarse una configuración adicional relacionada con los permisos de usuario. De manera predeterminada, los comandos Docker requieren privilegios administrativos y suelen ejecutarse mediante `sudo`. Para evitar esta necesidad, es posible agregar el usuario actual al grupo denominado **docker**. Los miembros de este grupo pueden utilizar Docker directamente desde la terminal sin necesidad de elevar privilegios en cada comando, simplificando considerablemente el trabajo diario del administrador o desarrollador.

Una vez completados estos pasos, el sistema queda preparado para crear, ejecutar y administrar contenedores Docker de manera local, permitiendo comenzar a trabajar con imágenes, redes, volúmenes y aplicaciones contenedorizadas.



Cierre de la clase y anticipo de la próxima

A lo largo de esta clase se realizó una introducción integral al ecosistema Docker y a los conceptos fundamentales que sustentan su funcionamiento. En primer lugar, se analizó el contexto en el que surgió esta tecnología, comprendiendo la relación existente entre Docker y la filosofía DevOps, así como la necesidad de disponer de entornos consistentes y reproducibles para el desarrollo y despliegue de aplicaciones.

Posteriormente, se estudiaron los principales componentes que conforman la plataforma, incluyendo Docker Engine, las imágenes, los contenedores y los registros de imágenes. También se abordó el concepto de contenedorización y las ventajas que ofrece respecto de los métodos tradicionales de distribución de software.

Se profundizó además en el funcionamiento de las imágenes Docker, su estructura basada en capas y el papel fundamental del Dockerfile como mecanismo para automatizar la construcción de entornos. Asimismo, se analizaron conceptos como Scratch, las etiquetas de versiones y los registros de imágenes, comprendiendo cómo Docker facilita la distribución y reutilización de aplicaciones.

En la parte práctica se presentaron los comandos básicos para la creación y administración de contenedores, permitiendo comprender cómo ejecutar, detener, inspeccionar y administrar aplicaciones dentro de Docker. Finalmente, se realizó una comparación entre contenedores y máquinas virtuales,

identificando las ventajas y limitaciones de cada enfoque y analizando los escenarios donde resulta conveniente utilizar una u otra tecnología.

Docker constituye actualmente una de las herramientas más importantes dentro de la infraestructura moderna, siendo ampliamente utilizada en entornos de desarrollo, plataformas de nube, arquitecturas basadas en microservicios y procesos de integración y despliegue continuo.

En la próxima clase se avanzará hacia el trabajo con **Docker Compose**, una herramienta que permite definir y administrar aplicaciones compuestas por múltiples contenedores. Esto permitirá desplegar servicios relacionados entre sí —como servidores web, bases de datos y aplicaciones backend— mediante una única configuración centralizada, simplificando significativamente la administración de entornos más complejos.